Arduino Calculator using RS232

Author: Michael Lombard

Keywords: Arduino, C++, Windows API, Serial Communication, RS232, Embedded Systems, Microsoft visual studio.

1. Abstract Summary

For this project, a calculator system was designed where an Arduino microcontroller performs mathematical operations requested from a PC console application through RS232 serial communication. The PC application was written in C++ using Microsoft Visual Studio, and sends expressions entered by the user to the microcontroller as ASCII strings.

The Arduino gets and parses the expression, performs the calculation, and returns the result to the PC. The system shows successful serial communication, error handling, and hardware–software interaction, while logging all transactions for debugging and errors.

2. Conceptual Design and Methodology

The goal of the project was to implement an Arduino-based calculator capable of performing basic arithmetic operations on two arguments via serial communication from a PC.

The PC application was developed in C++ using Microsoft Visual Studio. C++ was selected as the language because of its modular structure and easy string handling using the **std::string** library, simplifying expression input and communication.

The Arduino Uno was programmed in C. The microcontroller receives expressions from the PC, validates their format, performs the requested calculation, and returns the result.

I added an I2C 16×2 LCD display and status LEDs to inicate communication status and errors.

3. Technical Architecture

Communication between the PC and microcontroller is performed through the Windows serial interface, where I access the COM port as a file stream using the Windows API.

The port is opened using **CreateFileW**, and communication parameters such as baud rate, parity, and stop bits are configured using the Device Control Block (DCB) structure and applied with **SetCommState**. Parity bits provide basic error detection, and the **COMMTIMEOUTS** structure prevents the application from hanging if communication is interrupted.

The Arduino hardware has a 16×2 LCD connected through I2C using the LiquidCrystal_I2C and Wire libraries. A green LED indicates successful communication, while a red LED indicates errors. The microcontroller is powered through the USB port or via a DC jack.

The PC application was developed in Microsoft Visual Studio, while the micrcocontroller firmware was created using the Arduino IDE. This dual environment also ties with industrial practices where software and firmware are developed independently.

4. Programming Challenges & Solutions

One of my headaches involved compatibility between C++ strings and Windows API functions that require wide-character Unicode strings. To get around this, a custom **StringToWString** conversion function was included to convert standard strings into wide-character format for functions such as **CreateFileW**.

Another problem was inconsistent user input when configuring serial parameters. I got around this by normalizing characters using the **toupper()** function.

On the Arduino side, a parser was implemented to process incoming ASCII commands. Each command must begin with "**CALC**" before the expression, otherwise it won't calculate. The parser extracts two arguments and an operator (+, −, ×, ÷), then performs the calculation, and returns the result. Division-by-zero is detected and flagged as an error to ensure no crashes.

5. Testing and Validation

Communication reliability and calculation results were checked numerous times. Connection tests were done using multiple baud rates (4800 and 9600) to confirm proper communication between the PC and the Arduino.

Odd expressions were sent to check that both the Arduino and application correctly detected syntax and communication errors. Transaction logging was verified by confirming that all sent expressions and received results were stored in log.txt.

6. Conclusion

Integration between the console application and the Arduino is realised through this project. By joining a C++ console interface with microcontroller-based processing, the system provides reliable communication and separates the user input from the calculation side. The LCD and LED displays also enhance the visualisation of real-time embedded communication.

Project Repository:

https://github.com/electrogyver2000/Arduino-Calculator.git